

Sparse Competition during Training For the Emergence of Specialized Modules

BMVC 2026 Submission # 22

Abstract

Modularity in deep neural networks has been proposed as a means of improving both interpretability and training by promoting disentangled representations and reducing redundancy. In this work, we study the emergence of modular structure through competition dynamics between groups of neurons during training. We introduce a method that (i) preserves baseline accuracy, (ii) induces usage-based modularity by sparsely routing inputs to neuron groups, and (iii) encourages specialization of these modules, such that their activations are correlated with input classes. We evaluate the proposed approach on ImageNet-100 and show that it consistently yields well-defined modules without supervision. These modules capture a meaningful high-level structure in the data, with individual modules responding to semantic categories (e.g., dogs or vehicles). Our results suggest that competitive dynamics can serve as a simple mechanism for inducing functional modularity in standard architectures.

1 Introduction

Modular computation has long been viewed as a promising route towards more interpretable and reusable neural representations. Early work on Mixture-of-Experts and sparse routing [1, 2] proposed architectural solutions to enforce module boundaries. However, architectural separation does not guarantee that modules acquire meaningful functional roles. A neural network can be made of modules while still representing task-relevant information in a redundant or entangled way.

Recent mechanistic interpretability work [3, 4] reveals that networks do decompose computation into identifiable sub-circuits. However, these studies reveal the existence of an internal structure without asking whether it is well-organized. Are networks learning reusable functions or cluttered, redundant heuristics that happen to solve the task? The field's focus on benchmark performance obscures this distinction. A network can achieve high accuracy through either reusable localized computations or opaque distributed heuristics.

A parallel line of work suggests that the organization of representations depends not only on the task and architecture, but also on the training process. Open-ended evolutionary systems such as Picbreeder [5, 6] show that evolved networks can have reusable internal regularities and meaningful responses to weight perturbations [7]. Stanley et al. [8] recently went further through the Fractured Entangled Representation Hypothesis, that stipulates that neural networks trained conventionally with SGD will exhibit fragmented and entangled representations, whereas approaches like Picbreeder produce Unified Factored Representations (UFR), where capabilities are encoded with clearer boundaries and less redundancies. Our

work is motivated by the question: can standard gradient-trained networks be biased to-046
ward more factored, usage-specialized internal organization ? Our work tries to find answers047
through the creation of meaningful modules in neural networks.048

A natural approach is to enforce modularity directly. The recent work on clusterability049
[8] proposes adding a differentiable penalty to the loss that encourages weights to clus-050
ter within pre-defined module groups. This achieves structural modularity because weight051
matrices become concentrated within module boundaries. However, this comes at a cost.052
Enforcing clusterability damages the task accuracy and fails to induce semantic specializa-053
tion. Modules do not develop meaningful task-related roles. This results in a tradeoff: either054
accept reduced accuracy for modular weights, or abandon the structural approach. A funda-055
mental question remains: can we achieve both modularity and strong task performance?056

In this paper, we propose that modularity should emerge from learned routing, not from057
weight structure. We introduce a routing-based approach using module dropout with dual058
entropy objectives. One objective concentrates each input on few modules, while another059
encourages different inputs to use different modules. Modules compete for inputs, driv-060
ing specialization without explicit architectural constraints. On ImageNet-100, sparse top-1061
routing matches or exceeds baseline accuracy while inducing strong semantic specialization.062
Classes with the same coarse label are routed to the same modules roughly $3\times$ more often063
than expected by chance. This specialization also persists hierarchically: increasing module064
count refines partitions while preserving semantic coherence.065

These results reveal a deeper mechanistic insight. Dense training, where all modules are066
active during training, also produces semantically specialized modules. Yet these modules067
fail dramatically under sparse inference. A controlled ablation shows that sparse competitive068
training is essential for modules to remain functional under sparse evaluation. Sparse training069
forces modules to be individually deployable, whereas dense training creates co-adapted070
specialists that rely on each other. This distinction separates two phenomena: the emergence071
of modular structure from the emergence of useful sparse modularity. Training dynamics072
is as important as representation structure. Additionally, we extend our approach in several073
directions. We propose learning module widths through a shared capacity budget to create074
competition over resources that drives specialization. We apply routing to multiple layers075
simultaneously and reveal how semantic hierarchy emerges across depths.076

Our main contributions break down as follows:077

- We introduce Sparse Competitive Routing, a label-free routing objective that induces078
usage-based modularity through per-sample sharpness and population-level balance.079
080
081
- We show that the resulting modules acquire specialization, with module assignments082
carrying substantially more class information than without modules or with clustered083
baselines.084
085
- We demonstrate that sparse competitive training is mechanistically necessary for sparse086
deployability: dense specialization is not enough.087
088
- We analyze the semantic organization of learned modules across module counts and089
layers, showing that learned partitions refine hierarchically while preserving semantic090
coherence.091

2 Related Works

Most of the recent endeavors around modularity revolve around either interpretability or efficiency. The interpretability goal consists of decomposing a model into weakly interacting components makes its internal functioning easier to interpret and analyze, and the efficiency is through conditional computation. Early work on modular representations in deep neural networks [19] attempts to produce simplified description of networks through the use of similar connection patterns. Subsequent work formalized this approach through the notion of clusterability [2]: some parts of the network can be thought of as clusters because its parts are densely connected, while they are sparsely connected to the rest. Following this, [8] find a way to measure clusterability in a differentiable way, making it possible to train a neural network on this as an objective, with a loss that encourages non-interacting clusters. They show that this indeed produces modular components in fully connected layers, but find that these clusters do not reliably become more task-specialized than clusters in non-modular baselines. Our work addresses this gap: rather than enforcing modularity as a static property of the weight graph, we study whether sparse competitive routing can induce usage-based modules that are preferentially selected by different visual classes. Also, recent work [2] in controlled neural systems show that structural modularity alone is insufficient for functional specialization, and that specialization might depend on resource constraints and architecture. Our work aims to argue that an architecture-agnostic with usage-based modularity is sufficient to attain specialization in a neural network. The gradient routing approach [9] attempts to control the location of specific capabilities or information in the network through the manipulation of backpropagation. They use data-dependant masks to gradients to choose which part of the network will learn from which data. This allows for the localization of language-model capabilities for later ablation without damaging the accuracy of the model on the rest of its uses. Their work relies on the user-supplied information of "what goes where", which is different from our works, in which we do not assign specific classes or semantic groups to modules. Instead we study whether sparse competitive routing, complemented with an anti-collapse objective, without module-level supervision, are sufficient for semantic module usage to emerge.

As models scaled to billions and even more recently to trillions of parameters, a reliable way to scale has emerged in the architecture design space: Mixture-of-Experts (MoEs) [2, 22, 23]. MoEs consist of a routing function to activate only a subset of parameters for each input, allowing practitioners to increase model capacity without increasing the computation. In vision, V-MoE showed that sparse expert routing can be competitive with dense vision transformers while reducing inference compute [24], while Soft-MoE showed that it was possible to build a fully differentiable MoE to avoid hard token routing [23]. MoEs also emerged in efficient vision architecture like CNNs [20], showing that experts can be coupled with tightly vision-prior based architectures. Our goal is different from most MoE work: we want to explore the pertinence of adding experts as an architectural prior that would limit the network's exploration of the space and its expressivity, and instead show that a softer steering through the loss can make them emerge. This emergence of modules, devoid of supervision, is also interesting for interpretability reasons, to study what the learning dynamics are and how they evolve compared to ones with forced modularity through the architecture.

Finally, our work is related to mechanistic interpretability methods that seek to build or discover circuits from deep neural networks as human-understandable units [11, 9]. Our work does not provide new circuit-discovery algorithms, yet we think our contributions on training for emergent specialization of modules paves the way for building circuits in an unsupervised

yet human-understandable manner.

3 Method

We propose *Sparse Competitive Routing*, a training procedure that induces usage-based modularity through the loss in neural networks. Unlike structural modularity objectives, our loss does not directly act to form weakly interacting clusters, but steers the network towards using heavily only one module, and to not collapse to just one module. Only one module is selected at a routed layer, and our loss helps the network organize and distribute the load evenly. Importantly, the routing decision is not supervised by class labels. Labels enter only through the ordinary classification loss.

3.1 Modularizing a Layer

Let $h_\ell(x) \in \mathbb{R}^{d_\ell}$ be the activation of a routed layer $\ell \in \mathcal{R}$. We divide its coordinates into K_ℓ modules $\mathcal{G}_{\ell,1}, \dots, \mathcal{G}_{\ell,K_\ell}$. In the simplest case, these are equal-width contiguous blocks of neurons. This partition defines where modules are, but does not assign any semantic role to them.

For each module k , let $m_{\ell,k} \in \{0, 1\}^{d_\ell}$ be the corresponding binary mask. The activation supported by module k is

$$h_{\ell,k}(x) = m_{\ell,k} \odot h_\ell(x),$$

and the full activation decomposes as

$$h_\ell(x) = \sum_{k=1}^{K_\ell} h_{\ell,k}(x).$$

We route inputs using the activation energy of each module,

$$E_{\ell,k}(x) = \|m_{\ell,k} \odot h_\ell(x)\|_2 + \varepsilon, \quad (1)$$

where ε is a small numerical constant. Thus, a module is selected according to how strongly its neurons respond to the input. Unlike structural modularity objectives, which directly constrain the weight matrix, our weights remain unconstrained; modularity is induced through input-dependent usage. We later relax the equal-width block prior with the Gaussian-ring parameterization, explained at the end of the next subsection.

3.2 Sparse Competitive Routing Objective

The central goal of our method is to induce structured module usage without providing any class-related information or assignment. For each training sample (x_i, y_i) , the label y_i is used only through the usual classification loss. The routing objective does not observe y_i , it acts only on the activation patterns produced by the network itself. Thus, if visual classes become associated with a different modules, this structure has emerged from the usage-based competition between modules, without externally specified partition of the data.

Our objective combines two pressures. The first encourages each input to make a confident routing decision, localizing its computation to a small subset of modules. The second encourages balanced usage across inputs, ensuring that the network retains the capacity of

all modules. This balance is necessary because sparse routing has a natural collapse mode: all inputs may be routed to the same module. For k equally sized modules, this would effectively reduce the usable capacity of the routed layer to roughly $1/k$ of its original capacity. Collapse can be self-reinforcing: a slightly more responsive module receives more routed examples, hence more task gradients, which can further increase its dominance. Similarly, if one module initially learns a broadly useful classification strategy, the task loss alone provides a short-term incentive to route many inputs through that module. The routing objective must therefore make individual routing decisions sharp while keeping population-level module usage diverse.

For a routed layer ℓ , let $E_{\ell,k}(x_i)$ denote the activation energy of module k for input x_i , as defined above. We convert the energy profile into a differentiable routing distribution

$$q_{\ell,k}(x_i) = \frac{\exp(E_{\ell,k}(x_i)/\tau)}{\sum_{k'=1}^{K_\ell} \exp(E_{\ell,k'}(x_i)/\tau)}, \quad (2)$$

where $\tau > 0$ is a temperature parameter. Lower values of τ make the distribution closer to a winner-take-all decision, while higher values provide smoother gradients.

We minimize the entropy of the per-sample routing distribution to get the first pressure, where each sample has to select a very distinct module:

$$\mathcal{L}_{\text{sample}}^{(\ell)} = \frac{1}{B} \sum_{i=1}^B H(q_\ell(x_i)) = -\frac{1}{B} \sum_{i=1}^B \sum_{k=1}^{K_\ell} q_{\ell,k}(x_i) \log q_{\ell,k}(x_i), \quad (3)$$

where B is the batch size.

We then add the second pressure that maximize the entropy of the average usage distribution. This is a batch-level balance term, supposing that for a big enough batch of inputs, module usage should be uniform:

$$\mathcal{L}_{\text{bal}}^{(\ell)} = -H(\bar{q}_\ell) = \sum_{k=1}^{K_\ell} \bar{q}_{\ell,k} \log \bar{q}_{\ell,k}. \quad (4)$$

Equivalently, the objective can be viewed as encouraging high mutual information between inputs and module assignments, without using labels. The sharpness term lowers the conditional entropy of module assignment for each input, while the balance term raises the marginal entropy of module usage across inputs. This creates a pressure for confident but diverse assignments. The task loss then determines which such assignments are useful for prediction, allowing semantic specialization to emerge indirectly from training.

The final training objective is

$$\mathcal{L} = \mathcal{L}_{\text{task}} + \alpha(t) \mathcal{L}_{\text{route}}, \quad (5)$$

where $\mathcal{L}_{\text{task}}$ is the standard supervised loss and $\alpha(t)$ controls the strength of the routing objective during training.

Gaussian-ring module masks for capacity competition. The default SCM formulation partitions a layer into K equal-width contiguous blocks. This imposes a fixed-capacity prior: every module receives exactly the same number of neurons. To relax this assumption, we optionally replace hard block masks with soft Gaussian masks on a circular neuron axis.

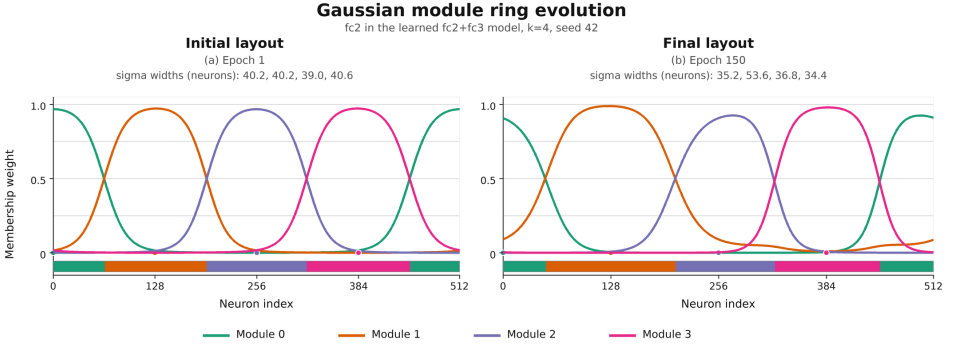


Figure 1: Illustration of the working of the gaussians ring, with a run where the layer fc2 with 512 neurons is modularized into 4 modules. Left is the first epoch, and right is the last epoch. We can see the network learned to get a dominant module 1, without total collapse.

Neuron i and module m are assigned positions $p_i = i/d_\ell$ and $c_m = m/K$ on the unit ring, and module memberships are computed as

$$G_{i,m} = \text{softmax}_m \left(-\frac{d_{\text{ring}}(p_i, c_m)^2}{2\sigma_m^2 T_{\text{gauss}}} \right), \quad (6)$$

where d_{ring} is the periodic distance and σ_m is a learnable module width. The widths are constrained by a fixed total budget, so increasing the capacity of one module reduces the capacity available to others. Routing energies are then computed exactly as before, replacing the hard module mask $m_{\ell,k}$ with the soft membership vector $G_{:,k}$.

This modification does not change the SCM objective or the sparse top-1 routing rule. It only replaces fixed equal-width modules with soft capacity-adaptive modules. As shown in Table 3, the Gaussian ring does not substantially affect accuracy, but consistently increases class-module mutual information. We therefore use SCM with Gaussian-ring masks in the remaining experiments.

3.3 Training and Inference

The neural network is trained end-to-end with standard SGD, and a hard top-1 mask during the forward, forcing the outputs of all the non-selected modules to be 0. Hence gradients obtained through the classification loss update only the selected module at that layer. The auxiliary routing loss is computed from the differentiable energy profile before the hard routing decision, allowing the model to reshape its activation energies during training to change the preferred module of a layer for a given input.

At inference time, we use the same sparse top-1 rule as during training:

$$z_\ell(x) = \arg \max_k E_{\ell,k}(x), \quad (7)$$

$$\tilde{h}_\ell(x) = m_{\ell,z_\ell(x)} \odot h_\ell(x). \quad (8)$$

Hence the model is trained in the same sparse regime in which it was trained: with top-1 selection. This match is important because dense modular training through usage-based

competition can yield class-structured energy profiles without producing modules that work independently from the rest of the layer. We therefore distinguish sparse competitive training from a dense control, shown in Figure 2-A in which all modules remain active during training and sparse top-1 routing is applied only at evaluation.

3.4 Usage-Based Modularity Metrics

Our approach is to, given the modules already separated like [8], steer the network towards an even and meaningful distribution of module selection. We want the routing to carry as much information about the specialized computation as possible. In a classification task, subtasks are classifications of superclasses, hence a specialized module should be detected by a module only being activated when inputs corresponding to classes of a given superclass are present.

We then, for each of the routed layer ℓ measure for the selected module Z_ℓ its class-module specialization using the mutual information

$$I(C; Z_\ell) = \sum_{c,k} p(c,k) \log \frac{p(c,k)}{p(c)p(k)} \quad (9)$$

4 Experiments

4.1 Setup

Backbone. For comparability with the clusterability baseline, we use the same global architecture as [8]: a small CNN stem followed by a bias-free fully connected head. After validating our method on the original CIFAR-10-scale setting, we scaled the network only enough to train from scratch on ImageNet-100. Our `scaled_cnn` maps $x \in \mathbb{R}^{3 \times 224 \times 224}$ through three Conv-BN-ReLU-MaxPool blocks with channels 32, 64, 128, then adaptive-average-pools to 4×4 , yielding a 2048-dimensional feature vector. The head is $2048 \rightarrow 512 \rightarrow 512 \rightarrow 512 \rightarrow 100$, with ReLU activations after the first three fully connected layers and no bias terms. We denote the three hidden fully connected layers by fc1, fc2, and fc3: fc1 maps the convolutional feature vector to the first 512-dimensional hidden representation, while fc2 and fc3 are 512-to-512 hidden transformations. Routing is applied only to selected layers among fc1, fc2, and fc3; the final 100-way classifier remains dense.

Dataset and training. Our main experiments are conducted with ImageNet-100, a subset of 100 classes of the 1000 classes of ImageNet. Instead of using a random subset, which could vastly change results, we use the standard CMC subset [18], favored to balance the classes over a wide and fair range of classes. All methods are evaluated with the same backbone and training protocol within each comparison. We modularize fully connected layers by partitioning their hidden units into equal-width modules. Unless stated otherwise, routing is applied to fc2 with $k = 4$ modules. We also report sweeps over routed layers (fc2, fc3, and fc2+fc3) and over the number of modules $K \in \{4, 8, 16\}$.

We compare Sparse Competitive Routing against two baselines. The vanilla baseline is trained with the standard classification loss and no modular objective. The clustered baseline follows the clusterability approach of [8], which encourages weights to concentrate within predefined module groups. Unless otherwise stated, SCM refers to sparse competitive routing with Gaussian-ring module masks.

Evaluation. For our method, training and inference use the same sparse top-1 routing rule: only the selected module remains active at a routed layer. We also include a dense-control variant, where the routing objective is used during training but all modules remain active in the forward pass. This separates semantic specialization of the routing scores from sparse deployability of the selected modules.

We report classification accuracy and class-module mutual information $I(Y; M_\ell)$, where M_ℓ is the selected module at layer ℓ . To detect collapse, we also report the effective number of used modules,

$$N_{\text{eff}} = \exp(H(M_\ell)).$$

For semantic analysis, we measure whether classes from the same coarse ImageNet group are assigned to the same module more often than expected by chance. In the module-count sweep, we additionally measure whether higher- K partitions refine lower- K partitions using pair retention and weighted child purity.

Finally, to test robustness to the choice of classes, we repeat the main ImageNet-100 experiment on two random class subsets. We report mean and standard deviation across matched seeds when available.

4.2 Results

Our method preserves accuracy with module specialization Table 1 reports the main ImageNet-100 CMC comparison for routing at fc2 with $k = 4$ modules. The vanilla model reaches $58.77 \pm 0.38\%$ accuracy, while the clustered structural-modularity baseline reaches $57.67 \pm 0.26\%$. Our method, evaluated in the sparse top-1 regime in which it is trained, obtains $57.84 \pm 0.59\%$, matching the clustered baseline and remaining within roughly one percentage point of the vanilla model. Thus, sparse competitive routing does not require the accuracy degradation typically associated with forcing a weight matrix to become structurally modular.

The main difference is not accuracy but the information carried by module usage. Vanilla and clustered models have low class-module mutual information at fc2, with $I(Y; M_{fc2}) = 0.036 \pm 0.006$ and 0.049 ± 0.012 , respectively. In contrast, our method reaches 0.649 ± 0.128 , an increase of approximately $18\times$ over vanilla and $13\times$ over the clustered baseline. This shows that the selected module is no longer an arbitrary subdivision of the layer: it becomes a class-informative routing variable.

This effect is robust to the choice of ImageNet-100 classes To test whether the CMC split contains an unusually favorable semantic structure, we repeat the experiment on two random ImageNet-100 label subsets. The results in Table 2 show the same qualitative pattern. Averaged across the two random subsets, sparse top-1 SCM obtains 59.55% accuracy, compared with 59.06% for vanilla and 59.17% for the clustered baseline. At the same time, SCM obtains a mean class-module mutual information of 0.5447, compared with 0.0636 for the clustered baseline. Thus, the specialization effect is not specific to the CMC label subset: across different class selections, sparse competitive routing preserves baseline-level accuracy while making module assignments substantially more predictive of class identity.

Increasing the number of modules gives finer specialization, gaussians ring too. Table 3 isolates the effect of the Gaussian-ring masks. Across $K \in \{4, 8, 16\}$, adding the ring changes top-1 accuracy only marginally, but increases $I(Y; M_{fc2})$ consistently. The

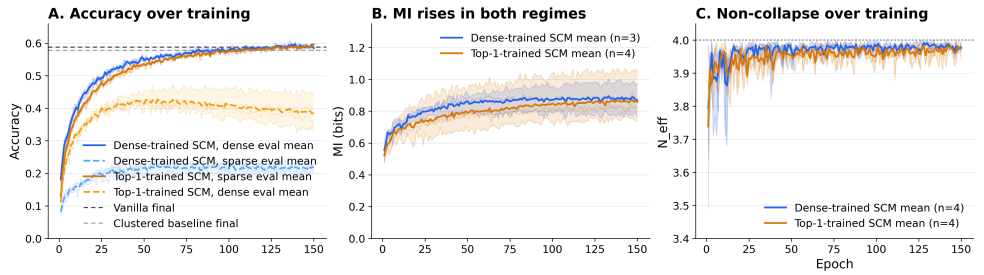


Figure 2: Evolution of metrics during trainings on ImageNet-100 CMC. A. The top-1 accuracies of our method (SCM) compared to no clustering (Vanilla) and the method from [2] (Clustered); in dotted lines are the opposite evaluations compared to training: the dense training is on sparse eval, and the sparse training is on dense eval. Both regimes fair better when evaluated like training, but sparse training fairs better on dense than dense on sparse eval. B. The evolution of MI between our method trained in sparse and dense training. We see that their MI ends up similar. C. The evolution of the effective number of modules at the end of training. It shows that our trainings don’t collapse: the trainings on our method converge on $N_{eff} = 4$ meaning all modules are used evenly.

ring should therefore be interpreted as a specialization-enhancing capacity parameterization, rather than as an accuracy-improving component.

The Gaussian ring does not substantially change sparse top-1 accuracy: across $K \in \{4, 8, 16\}$, the difference relative to SCM without the ring is small. However, it consistently increases $I(Y; M_{fc2})$, from 0.409 to 0.568 at $K = 4$, from 0.730 to 0.891 at $K = 8$, and from 1.150 to 1.297 at $K = 16$. We therefore use SCM with Gaussian-ring competition in the remaining experiments, not as an accuracy booster, but as a mechanism that strengthens modular specialization.

Sparse training is necessary for sparse deployability Figure 2-A and B separates two notions of modularity. A dense-control model can learn class-structured routing scores while all modules remain active in the forward pass. However, because every training example can rely on all modules, the selected module is not forced to be individually sufficient. Applying top-1 routing only at evaluation therefore produces modules that are semantically recognizable but not reliably deployable. In contrast, SCM trains and evaluates with the same hard top-1 mask. The selected module receives the task gradient for that example, while non-selected modules do not. This forces each routed module to support the computation assigned to it. The ablation therefore shows that semantic specialization of routing scores is not enough: sparse competitive training is required to obtain modules that remain functional under sparse inference.

Modules recover coarse semantic structure: the emergence of a hierarchy of classes We wanted to analyze the groupings of classes realized by our modules, making the hypothesis that classes from the same coarse group should be gathered to the same module much more often than chance (for example, all the dog classes gathered in the same module, so that this module has effectively become a dog classifier). We see from Figure 3 that our

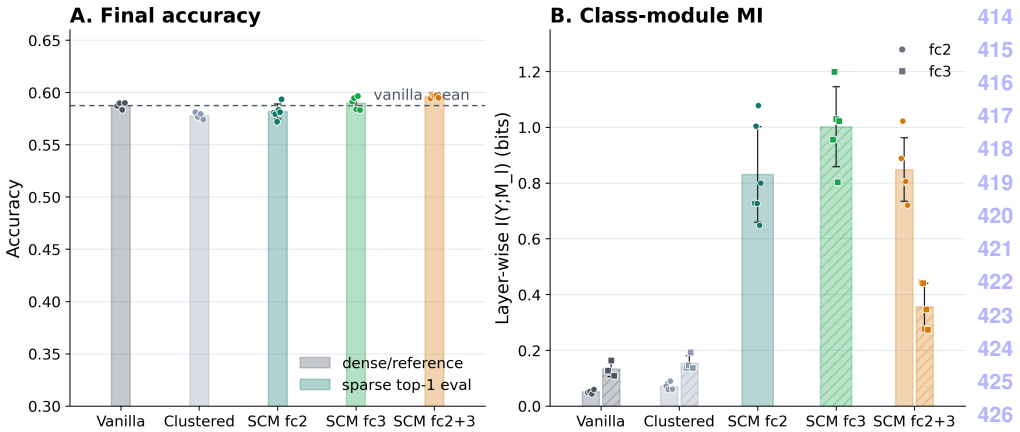


Figure 3: Measures of accuracies and MI of the same CNN model after a training on ImageNet-100 CMC. A: The accuracies of our method, clustered baseline from [8] and without modularization. Here, the accuracy of our method corresponds to the top-1 accuracy while other methods are evaluated on dense accuracies. B: the MI between the favored module and the input class.

Table 1: ImageNet-100 CMC comparison on matched seeds for the $fc2$ layer.

Method	Dense acc.	Sparse top-1 acc.	$MI(Y; M_{fc2})$
Vanilla	58.77 ± 0.38	—	0.036 ± 0.006
Clustered baseline	57.67 ± 0.26	—	0.049 ± 0.012
Ours	52.39 ± 0.33	57.84 ± 0.59	0.649 ± 0.128

4.3 More analysis

Causal specialization tests Finally, we test whether class-module associations are causally relevant rather than merely correlational. For each class, we identify its dominant module, ablate that module at test time, and compare the accuracy drop on classes owned by the ablated module with the drop on all other classes. Table 4 shows that SCM has positive causal gaps at both routed layers: 10.99 percentage points at $fc2$ and 18.15 at $fc3$. Importantly, SCM maintains nearly uniform ownership across the four modules, with ownership $N_{\text{eff}} = 3.99$ at both layers.

The clustered baseline also exhibits causal gaps, but its ownership is highly imbalanced, especially at $fc3$, where the effective number of class-owning modules drops to 1.71. Thus, SCM does not merely increase class-module mutual information; it induces a balanced decomposition whose modules are functionally relevant for their assigned classes. Since this diagnostic is run on a single seed, we report the full per-module breakdown in Appendix ??.

Routing deeper layers improves the accuracy-specialization tradeoff. We next ask where sparse competitive routing should be applied. Table 5 reports a single-seed layer-placement sweep on ImageNet-100 CMC using the same `scaled_cnn`, 150 epochs, and $K = 4$ modules. Routing at $fc1$ already induces substantial class-module specialization, increasing $I(Y; M)$ from 0.0179 to 0.4059, but it comes with a 3.04 percentage-point accuracy

Table 2: Results across two random ImageNet-100 label subsets. Sparse top-1 SCM preserves baseline-level accuracy while substantially increasing class-module mutual information. Bold denotes the best value within each metric group.

Subset	Accuracy				MI(class; module)	
	Vanilla	Clustered	SCM dense	SCM top-1	SCM	Clustered
Seed 42	0.5528	0.5528	0.5136	0.5628	0.5062	0.0614
Seed 43	0.6284	0.6306	0.5972	0.6282	0.5832	0.0657
Mean	0.5906	0.5917	0.5554	0.5955	0.5447	0.0636

Table 3: Module-count sweep and Gaussian-ring ablation for the routed fc2 layer on ImageNet-100 CMC. Acc. denotes top-1 accuracy. For SCM, accuracy is sparse top-1 accuracy; for the clustered baseline, accuracy is dense top-1 accuracy. Gaussian-ring competition does not substantially change accuracy, but consistently increases class-module mutual information.

K	Method	Acc. (%)	$I(Y; M_{fc2})$	N_{eff}
4	Clustered baseline	57.92 ± 0.22	0.048 ± 0.015	3.83 ± 0.02
4	SCM w/o Gaussian ring	57.53 ± 0.81	0.409 ± 0.091	3.96 ± 0.02
4	SCM + Gaussian ring	57.92 ± 0.22	0.568 ± 0.111	3.97 ± 0.01
8	Clustered baseline	58.74 ± 0.44	0.062 ± 0.012	7.83 ± 0.03
8	SCM w/o Gaussian ring	57.61 ± 0.46	0.730 ± 0.039	7.85 ± 0.05
8	SCM + Gaussian ring	57.79 ± 0.22	0.891 ± 0.050	7.83 ± 0.08
16	Clustered baseline	57.56 ± 0.87	0.113 ± 0.019	15.09 ± 0.08
16	SCM w/o Gaussian ring	55.77 ± 0.73	1.150 ± 0.030	15.31 ± 0.12
16	SCM + Gaussian ring	56.81 ± 0.38	1.297 ± 0.026	15.52 ± 0.10

drop relative to the matched dense baseline. This suggests that early fully connected features are still too general: forcing a sparse module choice at this stage removes useful shared computation.

Routing later layers gives a better tradeoff. At fc2, sparse top-1 routing slightly exceeds the matched baseline accuracy, 58.06% versus 57.92%, while increasing class-module mutual information from 0.0476 to 0.5049. Routing only fc3 gives the best single-layer specialization, with $I(Y; M) = 0.6628$, and also improves accuracy by 0.32 percentage points. Finally, routing both fc2 and fc3 gives the strongest matched result overall: sparse top-1 accuracy reaches 59.42%, 1.30 percentage points above the matched baseline, while both routed layers acquire nontrivial class information. This indicates that sparse competitive routing is most effective when applied to later representation layers, where features are already semantically organized enough for module selection to become class-informative.

Higher-resolution module partitions refine lower-resolution ones We ask next whether increasing k on completely unrelated runs produces unrelated partitions or a hierarchy of refinements that is compatible across seeds and module numbers: if modules meaningfully partition data on consistent granularity. Table 6 reports pair retention and weighted child purity across these three runs. Importantly, the $k = 4$, $k = 8$, and $k = 16$ runs are trained

Table 4: Causal specialization diagnostic on ImageNet-100 with four modules at fc2 and fc3. Classes are assigned to their dominant module; we then ablate each module and measure the accuracy drop on its owned classes and on all other classes. Drops are percentage points. Full per-module results are given in Appendix ??.

Model	Layer	Owned drop	Other drop	Gap	Ownership N_{eff}
SCM	fc2	22.38	11.39	10.99	3.99
SCM	fc3	27.32	9.17	18.15	3.99
Clustered	fc2	37.40	15.50	21.90	2.48
Clustered	fc3	24.48	10.59	13.89	1.71
Vanilla	fc2	32.22	21.64	10.58	3.39
Vanilla	fc3	19.90	13.13	6.77	3.32

Table 5: Layer-placement sweep on ImageNet-100 CMC. All rows use scaled_cnn, seed 42, 150 epochs, and $K = 4$ modules. Accuracy values are percentages.

Routed layers	Baseline acc.	Ours dense acc.	Ours top-1 acc.	Baseline MI	Ours MI
fc1	58.80	46.98	57.16	0.0179	0.4059
fc2	57.92	53.54	58.06	0.0476	0.5049
fc3	58.80	55.90	59.12	0.0763	0.6628
fc2+fc3	58.12	36.24	59.42	0.049 / 0.095	0.615 / 0.307

completely independently. We compare two independently trained partitions using two metrics. *Pair retention* is the fraction of class pairs assigned to the same module at lower k that remain coassigned at higher k . We compare it to a random baseline obtained by permuting the higher- k assignments while preserving module sizes. *Weighted child purity* measures whether each higher- k module mostly comes from a single lower- k parent, weighted by the number of classes in the child module. Table 6 shows that SCM obtains a clear refinement structure. From $k = 4$ to $k = 8$, pair retention is 28.32%, compared with $13.85 \pm 0.86\%$ under the random baseline. From $k = 8$ to $k = 16$, it gets 27.15, much higher than the random $6.40 \pm 0.89\%$. These fair better than the clustered baseline, being closer or even below their random baseline, and lower than our method. This is also true for weighted child purity, where we reach higher than clustered baseline, with 69.00 and 72.00 against their 66.00 and 48.00. The alluvial visualization in Figure 4 illustrates the same pattern: SCM modules at higher resolution behave like semantic subdivisions of lower-resolution modules, whereas clustered partitions are less stable across independently trained resolutions.

5 Conclusion

In this work we proposed a method that modularizes modules through competition. The competition is mainly carried through usage-based competition with Sparse Competitive Routing allowing for a modularization that produces meaningfully specialized modules. Our second, complementary way to further increase specialization of modules is through capacity-based competition with gaussian ring ownership of neurons, where the width of modules is learned. These two ways, as we showed allow for modules that conserve the accuracy of

Table 6: Hierarchy metrics for the `fc2` module-count sweep on ImageNet-100 CMC.

Method	Transition	Pair retention	Random baseline	Weighted child purity
Ours	4 $\rightarrow$ 8	28.32	13.85 $\pm$ 0.86	69.00
Ours	8 $\rightarrow$ 16	27.15	6.40 $\pm$ 0.89	72.00
Clustered	4 $\rightarrow$ 8	21.70	23.72 $\pm$ 1.55	66.00
Clustered	8 $\rightarrow$ 16	11.33	9.62 $\pm$ 0.83	48.00

Pair retention is the fraction of class pairs sharing a module at the lower count that remain coassigned at the higher count. The random baseline is the permutation baseline reported by the exported paper-figure artifact.

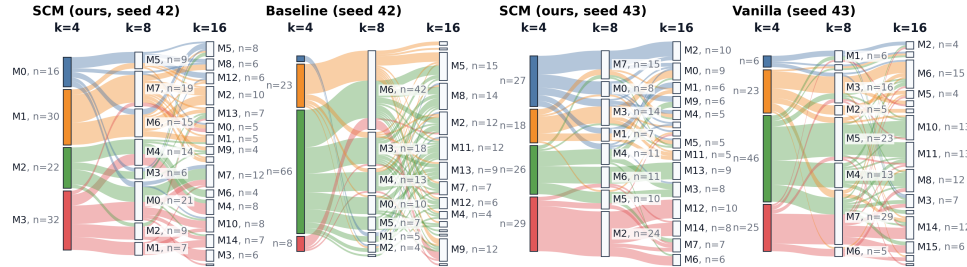


Figure 4: Alluvial visualization of class assignments across independently trained `fc2` runs with $K = 4, 8, 16$ modules. Each block is a learned module and each flow tracks ImageNet-100 classes across module-count resolutions. We compare the cleaner refinement structure obtained by our method against the two alternative flows shown in the figure. To rule out this structure being a seed artifact, we also show two independent seeds of our method, both of which exhibit similar coarse-to-fine class flows.

a non-modularized network, while meaningfully gathering the data into groups at varying granularity, adapting to its number of modules. This work demonstrates a new way to modularize networks: through the loss only, via information theoretic metrics. This approach further manifests that internal organization does not need to be monitored, but given the right incentives, it can appear spontaneously.

6 Limitations and future works

6.1 Limited Experiment Sizes

Our experiments are intentionally conducted in a controlled setting, which makes module usage, specialization, and ablations easy to measure. However, this also limits the scope of our conclusions. The main results use ImageNet-100 and a relatively small `scaled_cnn` backbone, with routing applied mostly to fully connected layers. Thus, our results show that sparse competitive routing can induce functional modularity in standard networks, but they do not yet establish that the same behavior scales unchanged to larger architectures or datasets.

A second limitation is that only a small number of layers are modularized. Most experiments route one layer, and our multi-layer results are limited to `fc2` and `fc3`. This is enough to show that module assignments become class-informative and remain usable under sparse

inference, but not enough to claim full-network modular computation or complete circuit recovery. Larger experiments with many routed layers are needed to test whether the learned modules form stable, causally meaningful circuits across depth.

6.2 Future Work

The most immediate direction is to test whether sparse competitive routing scales beyond the controlled vision setting studied here. Our experiments use relatively small fully connected layers on ImageNet-100, which makes module usage and class-level specialization easy to measure. A natural next step is to apply the same objective to larger backbones, larger vision datasets, and models with richer internal structure, such as vision transformers, language models, and vision-language models. These settings would test whether usage-based modularity can emerge in models where features are more distributed, tasks are less class-discrete, and semantic structure is not limited to a fixed set of visual labels.

A second direction is to modularize more layers simultaneously. In this work, most of our analysis focuses on one or two routed fully connected layers. However, the main interpretability promise of modular training is not only to obtain specialized modules at a single layer, but to obtain *circuits*: connected chains of modules that support particular computations across depth. Our preliminary multi-layer experiment in the Appendix, where fc2 is routed into four modules and fc3 into eight modules, suggests that this may occur naturally. In that setting, adding an explicit prior that each fc2 module should connect only to two dedicated fc3 modules produces little difference compared to leaving the inter-layer module connections unconstrained. This suggests that sparse competitive routing may already bias the network toward structured module-to-module pathways.

References

- [1] Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermyn, Tom Conerly, Nick Turner, Cem Anil, Carson Denison, Amanda Askell, Robert Lasenby, Yifan Wu, Shauna Kravec, Nicholas Schiefer, Tim Maxwell, Nicholas Joseph, Zac Hatfield-Dodds, Alex Tamkin, Karina Nguyen, Brayden McLean, Josiah E Burke, Tristan Hume, Shan Carter, Tom Henighan, and Christopher Olah. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023. <https://transformer-circuits.pub/2023/monosemantic-features/index.html>.
- [2] Gabriel B  na and Dan F. M. Goodman. Dynamics of specialization in neural modules under resource constraints. *Nature Communications*, 16(1):187, 2025. doi: <https://doi.org/10.1038/s41467-024-55188-9>. URL <https://www.nature.com/articles/s41467-024-55188-9>.
- [3] Alex Cloud, Jacob Goldman-Wetzler, Ev  zen Wybitul, Joseph Miller, and Alexander Matt Turner. Gradient routing: Masking gradients to localize computation in neural networks. *arXiv preprint arXiv:2410.04332*, 2024.
- [4] Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models. *arXiv preprint arXiv:2309.08600*, 2023.

- [5] Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, et al. Toy models of superposition. *arXiv preprint arXiv:2209.10652*, 2022.
- [6] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *ArXiv*, abs/2101.03961, 2021. URL <https://api.semanticscholar.org/CorpusID:231573431>.
- [7] Daniel Filan, Stephen Casper, Shlomi Hod, Cody Wild, Andrew Critch, and Stuart Russell. Clusterability in neural networks, 2021. URL <https://arxiv.org/abs/2103.03386>.
- [8] Satvik Golechha, Maheep Chaudhary, Joan Velja, Alessandro Abate, and Nandi Schoots. Studying cross-cluster modularity in neural networks, 2025. URL <https://arxiv.org/abs/2502.02470>.
- [9] Joost Huizinga, Kenneth O Stanley, and Jeff Clune. The emergence of canalization and evolvability in an open-ended, interactive evolutionary system. *Artificial life*, 24(3): 157–181, 2018.
- [10] Robert A Jacobs, Michael I Jordan, Steven J Nowlan, and Geoffrey E Hinton. Adaptive mixtures of local experts. *Neural computation*, 3(1):79–87, 1991.
- [11] Akarsh Kumar, Jeff Clune, Joel Lehman, and Kenneth O Stanley. Questioning representational optimism in deep learning: The fractured entangled representation hypothesis. *arXiv preprint arXiv:2505.11581*, 2025.
- [12] Dmitry Lepikhin, HyounJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. {GS}hard: Scaling giant models with conditional computation and automatic sharding. In *International Conference on Learning Representations*, 2021. URL <https://openreview.net/forum?id=qrwe7XHTmYb>.
- [13] Joan Puigcerver, Carlos Riquelme, Basil Mustafa, and Neil Houlsby. From sparse to soft mixtures of experts, 2023.
- [14] Carlos Riquelme, Joan Puigcerver, Basil Mustafa, Maxim Neumann, Rodolphe Jenatton, André Susano Pinto, Daniel Keysers, and Neil Houlsby. Scaling vision with sparse mixture of experts. In *Neural Information Processing Systems*, 2021. URL <https://api.semanticscholar.org/CorpusID:235417196>.
- [15] Jimmy Secretan, Nicholas Beato, David B D Ambrosio, Adelein Rodriguez, Adam Campbell, and Kenneth O Stanley. Picbreeder: evolving pictures collaboratively online. In *Proceedings of the SIGCHI conference on human factors in computing systems*, pages 1759–1768, 2008.
- [16] Jimmy Secretan, Nicholas Beato, David B D’Ambrosio, Adelein Rodriguez, Adam Campbell, Jeremiah T Folsom-Kovarik, and Kenneth O Stanley. Picbreeder: A case study in collaborative evolutionary exploration of design space. *Evolutionary computation*, 19(3):373–403, 2011.

- [17] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc V. Le, Geoffrey E. Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *ArXiv*, abs/1701.06538, 2017. URL <https://api.semanticscholar.org/CorpusID:12462234>.
- [18] Yonglong Tian, Dilip Krishnan, and Phillip Isola. Contrastive multiview coding. In Andrea Vedaldi, Horst Bischof, Thomas Brox, and Jan-Michael Frahm, editors, *Computer Vision – ECCV 2020*, pages 776–794, Cham, 2020. Springer International Publishing. ISBN 978-3-030-58621-8.
- [19] Chihiro Watanabe, Kaoru Hiramatsu, and Kunio Kashino. Modular representation of layered neural networks. *Neural Networks*, 97, 03 2017. doi: 10.1016/j.neunet.2017.09.017.
- [20] Yihua Zhang, Ruisi Cai, Tianlong Chen, Guanhua Zhang, Huan Zhang, Pin-Yu Chen, Shiyu Chang, Zhangyang Wang, and Sijia Liu. Robust mixture-of-expert training for convolutional neural networks. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 90–101, October 2023.

Appendix

Unless otherwise stated, all additional analyses use the same ImageNet-100 `scaled_cnn` setup as in the main paper. We report two complementary diagnostics. First, we visualize whether modular assignments across two routed layers form coherent cross-layer structure. Second, we provide the full per-module breakdown of the causal specialization test summarized in the main paper.

A. Cross-layer modular structure

When more than one layer is routed, a natural question is whether modules at different layers form coherent multi-layer circuits, or whether their assignments are independent across layers. To probe this, we compare two variants with a routed `fc2` layer using $K = 4$ modules and a routed `fc3` layer using $K = 8$ modules. In the unconstrained variant, no explicit relation is imposed between the modules of the two layers. In the constrained variant, we impose a hard hierarchical connectivity prior: each `fc2` module is allowed to feed only two designated `fc3` modules.

Figure 5 shows the resulting alluvial diagrams. Each flow represents classes that share the same dominant module at `fc2` and `fc3`. The constrained and unconstrained variants produce qualitatively similar flows, suggesting that cross-layer modular organization can arise without explicitly imposing a parent–child module hierarchy. This diagnostic is qualitative, but it supports the view that SCM can align modular assignments across layers through the training objective itself, rather than through a hand-designed connectivity prior.

B. Per-module causal specialization

The main paper reports an aggregate causal specialization diagnostic. Here we provide the full per-module breakdown. For each trained model and each routed layer, we first assign every class to its dominant module. We then ablate one module at test time and measure the accuracy drop on the classes owned by the ablated module and on all remaining classes. A positive gap indicates that the ablated module is more important for the classes assigned to it than for other classes.

Table 7 shows that SCM has a positive causal gap for every module at both `fc2` and `fc3`. Moreover, SCM assigns classes nearly uniformly across modules: each module owns between 23 and 28 classes. The clustered baseline also exhibits positive causal gaps, but with substantial ownership imbalance. In particular, at `fc3`, one clustered module owns 86 out of 100 classes, while the other three modules own only 4, 8, and 2 classes. Vanilla networks also show positive gaps, but the effect is weaker at `fc3`. These results support the interpretation that SCM induces balanced, causally relevant specialization rather than only class-correlated routing assignments.

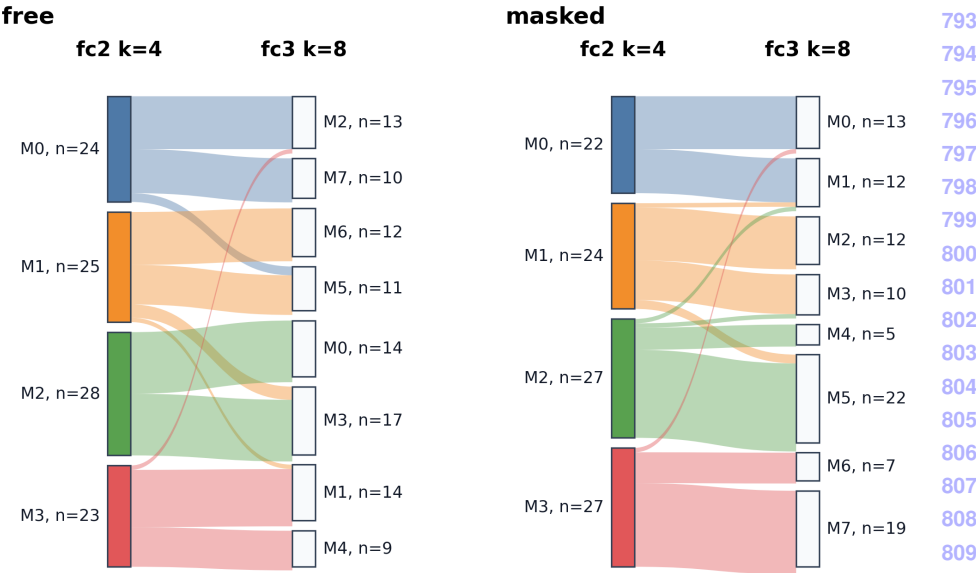


Figure 5: Alluvial visualization of shared dominant classes between a routed $fc2$ layer with $K = 4$ modules and a routed $fc3$ layer with $K = 8$ modules. We compare an unconstrained model with a hierarchy-constrained variant in which each $fc2$ module can connect only to two designated $fc3$ modules. The two diagrams are qualitatively similar, indicating that cross-layer modular structure can arise without explicitly enforcing a hierarchical routing prior.

Table 7: Per-module causal specialization results on ImageNet-100 using the `scaled_cnn`, seed 42, and four-module `fc2+fc3` runs. Each row ablates one module. Classes are assigned to their dominant module before ablation. Drops and gaps are reported in percentage points.

Model	Layer	Module	Owned classes	Owned drop	Other drop	Gap
SCM	fc2	1	25	20.96	8.24	12.72
SCM	fc2	2	23	23.30	13.22	10.08
SCM	fc2	3	24	20.00	14.39	5.61
SCM	fc2	4	28	24.93	9.53	15.40
SCM	fc3	1	24	23.83	11.95	11.89
SCM	fc3	2	25	25.04	9.65	15.39
SCM	fc3	3	23	25.04	9.38	15.67
SCM	fc3	4	28	34.21	5.53	28.69
Clustered baseline	fc2	1	12	51.83	15.89	35.95
Clustered baseline	fc2	2	16	46.75	18.12	28.63
Clustered baseline	fc2	3	3	67.33	12.31	55.02
Clustered baseline	fc2	4	69	31.42	17.29	14.13
Clustered baseline	fc3	1	4	38.00	12.23	25.77
Clustered baseline	fc3	2	8	50.25	12.43	37.82
Clustered baseline	fc3	3	2	44.00	8.08	35.92
Clustered baseline	fc3	4	86	21.00	4.86	16.14
Vanilla	fc2	1	39	25.03	17.74	7.29
Vanilla	fc2	2	38	38.74	26.87	11.87
Vanilla	fc2	3	15	34.80	24.78	10.02
Vanilla	fc2	4	8	31.50	17.80	13.70
Vanilla	fc3	1	34	20.06	13.97	6.09
Vanilla	fc3	2	10	21.20	15.42	5.78
Vanilla	fc3	3	11	16.18	11.19	4.99
Vanilla	fc3	4	45	20.40	11.49	8.91